

Quiz 4: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 4 is Thursday Sep 24, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Scope: the second half of backpropagation

Quiz 3 covered backpropagation up to “each node needs to know two things.” This quiz picks up from exactly that point and finishes the story.

On the quiz:

- Local derivatives at a node, and the rules for add, multiply and ReLU
- Working a backward pass through a graph and reading off the numbers
- **Adding gradients where two paths meet**
- Topological order: what it is for, and what breaks without it
- Reverse mode against forward mode, and why a scalar cost decides it
- The sigmoid and cross-entropy cancellation, $\partial \ell / \partial z = \hat{y} - y$
- The **three gradients** a layer owes, and what each is for
- Shapes, and why nobody forms a Jacobian
- Why training needs more memory than inference
- Exploding and vanishing gradients, and where in the network they bite

On the quiz from Lab 3, which was due the day before:

- Why the `Value` class accumulates with `+=` rather than assigning
- Why the backward pass needs a seed, and why the seed is 1.0
- What the forward-pass count comparison showed: **20,001 against 401**

Not on this quiz: anything in PyTorch. `.backward()`, `zero_grad()`, `nn.Module`, optimizers and the training loop are Quiz 5. If it has a `torch.` in front of it, it is not here.

Format. Same shape as Quizzes 2 and 3: true/false, multiple choice and multiple answer for 20 of the 25 points, then two short written questions.

Part 1: The backward pass on a graph

Local derivatives at a node

A node does not know anything about the network. It knows its own operation, and it gets handed a number from downstream. Write g for that incoming gradient, $\partial\mathcal{L}/\partial(\text{this node's output})$.

Node	Backward behaviour
$c = a + b$	a and b each receive g . An add copies the gradient.
$c = a \times b$	a receives bg and b receives ag . A multiply swaps the inputs in.
$c = \max(0, a)$	a receives g if $a > 0$, otherwise 0. A ReLU is a gate .
$c = f(a)$	a receives $f'(a)g$.

Notice that every rule uses the **forward** values. That is why they have to be kept, and why training a network uses much more memory than running one.

Gradients sum over paths

This is the single most important fact of the week. If a node's output feeds two later nodes, its gradient is the **sum** of the contributions coming back along both paths. It is the multivariable chain rule:

$$\frac{\partial\mathcal{L}}{\partial u} = \sum_k \frac{\partial\mathcal{L}}{\partial v_k} \frac{\partial v_k}{\partial u}$$

Two consequences worth being able to state:

1. In code, a node **accumulates** into its gradient with `+=` and never with `=`, because you do not know whether another path is still to arrive. That is the `+=` in Lab 3's `Value` class.
2. It is why PyTorch adds into `.grad` instead of overwriting, and therefore why `zero_grad()` exists. (The PyTorch half is Quiz 5; the reason is here.)

Practise the computation, not just the slogan. Take $a = 3$, $b = 4$ and

$$c = a \times b \quad d = c + a \quad e = d \times b$$

Both a and b are used twice. Work backward and you should get $\partial e/\partial a = 20$ and $\partial e/\partial b = 27$. If you get 16, you took the path through c and forgot the path through d . Check yourself by substituting: $e = a(b^2 + b)$.

Topological order

The backward pass has to visit a node **only after** everything it feeds has already been visited, which is reverse topological order. Kahn's algorithm is how you get that order: repeatedly take a node with no unprocessed dependencies, and remove it.

What you need for the quiz is the **reason**, not the pseudocode. If you process a node too early, it collects only part of its gradient, because a contribution from some downstream node that has not run yet never gets added. The number it ends up with is wrong, silently, and with a plausible-looking magnitude.

This is also why the graph must be acyclic: a cycle has no valid order.

Seeding the pass

The pass begins by setting the output node's gradient to 1.0. Two things to be able to say:

- **Why that value:** the pass computes $\partial\mathcal{L}/\partial(\text{node})$ for every node, and for the cost node itself that is $\partial\mathcal{L}/\partial\mathcal{L} = 1$.
- **Why a seed at all:** every other gradient is a product of local derivatives against a value arriving from downstream. The chain has to start somewhere, and seeding at 1 is what makes each later product mean “derivative of the cost with respect to this node.”

Part 2: Reverse mode, and why it is the right direction

Forward mode against reverse mode

Both compute exact derivatives by the chain rule. They differ in the direction they sweep, and therefore in what one sweep buys you:

	One sweep gives you	Cost scales with
Forward mode	one input's effect on every output	number of inputs
Reverse mode	every input's effect on one output	number of outputs

A deep network has millions of inputs, the parameters, and exactly **one** output, the cost. So reverse mode needs one sweep and forward mode would need one per parameter. **The fact that the cost is a scalar is what decides it.** That is the examinable sentence.

Flip it around and the answer flips: many outputs and one input would favour forward mode.

What one backward pass buys

One forward pass plus one backward pass gives $\partial\mathcal{L}/\partial w$ for **every** parameter at once, and the backward pass costs roughly what the forward pass costs regardless of the parameter count.

Compare against finite differences, which need $n + 1$ forward passes and are only **approximate**. Backprop wins on both axes. Lab 3 measured exactly this: **20,001 forward passes against 401** for the same fit, with agreement to about 10^{-10} against central differences.

The sigmoid and cross-entropy cancellation

With a sigmoid output and binary cross-entropy, the derivative of the loss and the derivative of the activation cancel, and the gradient at the pre-activation z collapses to

$$\frac{\partial\ell}{\partial z} = \hat{y} - y$$

Just the prediction error. This is why the pairing is standard: the gradient is numerically stable and stays large when the prediction is badly wrong, instead of being throttled by σ' . Be able to state the result and say why the pairing is chosen; you will not be asked to rederive it from scratch.

Part 3: From a scalar to a layer

The three gradients

A layer computes $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ and receives $\partial\mathcal{L}/\partial\mathbf{z}$ from the layer in front. It owes three things:

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}} = \frac{\partial\mathcal{L}}{\partial\mathbf{z}}\mathbf{x}^\top \quad \frac{\partial\mathcal{L}}{\partial\mathbf{b}} = \frac{\partial\mathcal{L}}{\partial\mathbf{z}} \quad \frac{\partial\mathcal{L}}{\partial\mathbf{x}} = \mathbf{W}^\top \frac{\partial\mathcal{L}}{\partial\mathbf{z}}$$

Know the split, not just the formulas. The first two are what the optimizer updates. The third is the upstream gradient handed to the layer **behind** this one, and it is what makes the pass propagate at all. A student who writes all three but cannot say which two get updated has missed the question.

Nothing here inverts anything. It is two matrix multiplies.

Shapes

A gradient has the **same shape** as the thing it is a gradient of, because there is one partial derivative per entry. A 3×2 weight matrix has a 3×2 gradient. This is also the debugging trick: if your backward pass produces a transpose of the shape you expected, you have to multiply the wrong way round.

Why nobody forms a Jacobian

For a batch of 128 with 128 features in and 256 units out, the full Jacobian of one layer is about **6.5 GB** in float32. The gradients training actually needs come to about **198 KB**.

Reverse mode never builds the big object. It only ever computes the incoming gradient multiplied through by $\mathbf{W}^\top$ or $\mathbf{x}^\top$, an ordinary matrix product whose result is shaped like the parameters. The full matrix of every output coordinate against every input coordinate is never materialized, because nothing in the algorithm needs it.

Exploding and vanishing gradients

The chain rule multiplies one factor per layer. Multiply many numbers below 1 and the product goes to zero; multiply many above 1 and it blows up.

The sigmoid is the classic offender: σ' never exceeds **0.25**, so ten sigmoid layers multiply an incoming gradient by at most $0.25^{10} \approx 10^{-6}$. The early layers receive almost nothing and stop learning.

Know where it bites. The layers **nearest the input** are worst affected, because their gradient has travelled through the most factors. Depth **lengthens** that chain; it does not shorten it.

This is a **training** problem, not an autodiff problem. Backprop hands you a correct gradient; it is just too small to be useful. The cures, ReLU, normalization, residual connections and careful initialization, are Week 5.

Checklist

Before Thursday, confirm you can:

- ☐ Give the backward rule for an add, a multiply and a ReLU node
- ☐ Work a backward pass by hand on a small graph and get the numbers right
- ☐ **Handle a node whose output feeds two places, and say why you add**
- ☐ Say why the code uses += and not =
- ☐ Say what topological order is for, and what goes wrong without it
- ☐ Say why the output node is seeded at 1.0, and why a seed is needed
- ☐ State what one sweep of forward mode and of reverse mode each buy
- ☐ Say why a scalar cost makes reverse mode the right choice
- ☐ Say what one forward and one backward pass give you, for n parameters
- ☐ State $\partial\ell/\partial z = \hat{y} - y$ and why that pairing is used
- ☐ Write the three layer gradients, and say which two the optimizer updates
- ☐ Give the shape of $\partial\mathcal{L}/\partial\mathbf{W}$
- ☐ Say why reverse mode never forms a Jacobian
- ☐ Explain vanishing gradients, and say **which layers** suffer most
- ☐ Arrive with a calculator and nothing else; no notes are permitted